

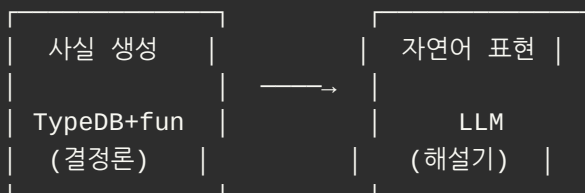
1. 개요 — 5주차 발표 3 한눈에 보기

본 자료는 5주차 스터디 발표 3 "온톨로지 + LLM 연동" 의 압축 요약본 입니다.
4개 PDF + (코드 zip 동봉 시) 5번째 PDF 로 구성됩니다.

한 줄 메시지

사실은 그래프와 함수가, 표현은 LLM 이.
정책은 노드로 외부화. 측정·자가수정은 시스템이.

역할 분담 한눈에



- ✗ LLM 은 사실 생성기가 아니다
- ✓ LLM 은 사실 해설기다

추론 모델 진화 3단계

구분	As-is (Python)	To-be (fun 하드코딩)	★ 본 스터디 (fun + 외부화) ★
자족성 (그래프가 직접 추론)	✗	✓	✓
외부화 (코드 없이 정책 변경)	✓	✗	✓

둘의 약점을 동시에 해소.

6대 Takeaway

#	메시지	시연
①	write-back : 추론 결과가 다시 데이터가 된다	시연 1
②	외부화 : 정책 변경 = INSERT 한 줄, 코드 0줄	시연 2
③	advisory : 감사 정보가 자연어에 묻어 나온다	시연 3
④	환각 방지 : DB 에 없으면 LLM 도 답을 만들지 않는다	시연 3·4
⑤	Self-correcting : 자가 수정 + 정량 측정 가능	시연 4
⑥	★ 자족 추론 : 그래프가 직접 추론할 수 있다	시연 2

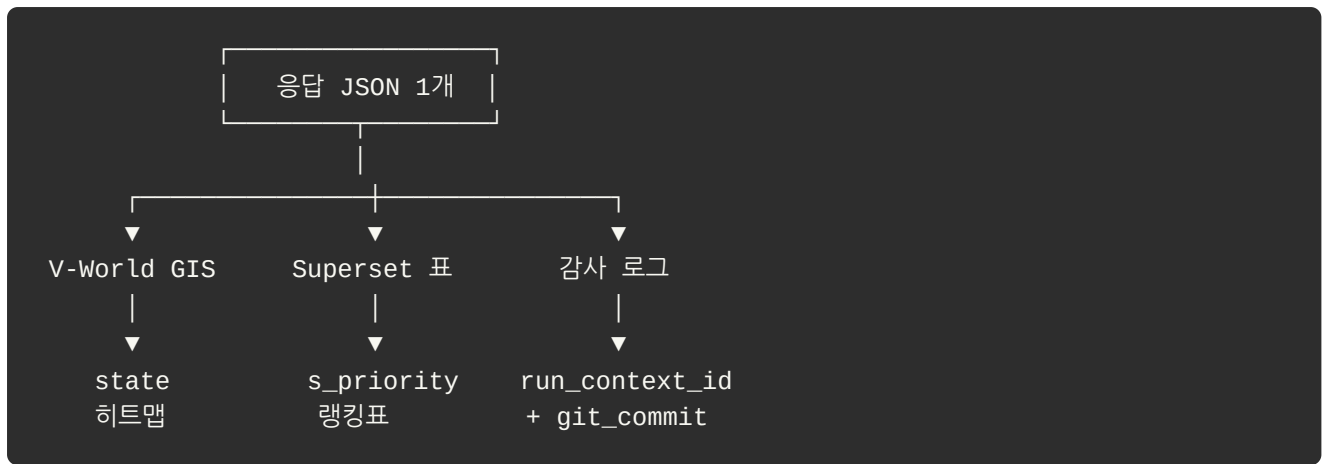
시연 4개 — 한 줄 소개

시연	무엇을 보여주는가	핵심 메시지
① write-back	source 29 + segment 5 위에서 추론 후 점수·State 를 그래프에 다시 기록	추론 결과가 다시 데이터가 된다
② fun 듀얼 ★	정책 노드(v1↔v2) 만 바뀌 결과 격상 + Python 추론 = fun 추론 100% 일치	코드 0줄 정책 시뮬레이션 + 그래프 자족 추론
③ chat.py + advisory	자연어 질의 3건. MOCK 인용 시 자동 advisory · 데이터 없으면 거부	감사 정보 + 환각 방지
④ 평가셋 14건	자동 평가. 직접성공 / 자가수정 / fallback / 실패 카운트	측정 가능 + Self-correcting

→ 상세 화면별 가이드는 [4_시연_요약.md] 에서.

응답 JSON = 모든 출력의 단일 원천

추론 결과 JSON 1개가 세 곳에 동시에 흘러간다.



본 자료 묶음 (총 4개 + 코드 zip 시 5개)

#	파일	한 줄 요약
1	이 문서 (개요)	한 줄 메시지 + 6대 Takeaway + 시연 4개 한 줄 소개
2	[2_데이터_와_엔티티.md]	source 29건 + 5종 엔티티 + 적재 38개 노드
3	[3_추론_과_LLM.md]	TypeQL fun · Write-back · 사용자 질의 흐름
4	[4_시연_요약.md]	4개 시연의 화면별 가이드
5	[5_코드_안내.md]	(코드 zip 동봉 시) 폴더 트리 + 시연↔코드 매핑

💡 한 줄 요약

"한 줄 메시지 → 6대 Takeaway → 4개 시연 → 응답 JSON 단일 원천. 이 한 체인이 5주차 발표 3의 전부."

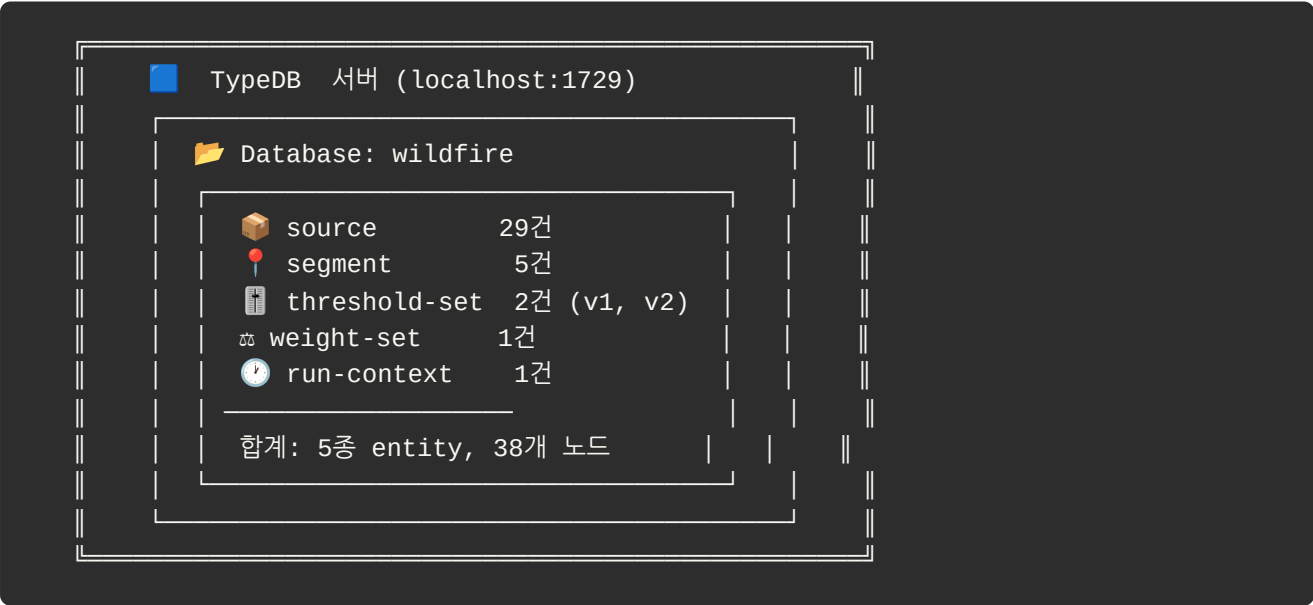
2. 데이터 와 엔티티

본 시스템이 무엇을 데이터로 보는가 를 한 곳에 정리. TypeDB 안의 5종 엔티티와 총 38개 노드의 의미.

📖 용어 풀이 (자료 곳곳에서 등장)

약어 / 용어	풀이
EMD_	읍·면·동 (Eup-Myeon-Dong). 한국 행정구역 표준 약어. 예: EMD_여수_상암동 = 여수시 상암동
S = 0.534	s-priority 의 약자. 종합 위험 점수 (0 ~ 1 범위)
TypeQL	TypeDB 의 쿼리 언어. SQL 이 관계형 DB 의 쿼리 언어인 것과 같은 관계
advisory	MOCK 데이터 인용 시 LLM 답변에 자동으로 따라붙는 "추정값입니다" 같은 경고문
fun	TypeDB 3.0 의 함수 기능. 판단 로직을 DB 안에 저장 (자세한 건 [3_추론_과_LLM.md] § 1)

1. TypeDB 안의 5종 엔티티 — 한 장 시각화



29 + 5 + 2 + 1 + 1 = 38개 노드

2. 각 엔티티의 비유와 역할

엔티티	비유	역할
 source	원산지 표기	데이터별 신뢰도(REAL/MOCK/EXCLUDE) 기록
 segment	환자 종합 차트	입력 + 추론 결과(write-back) 한 노드에 통합
 threshold-set	합격 커트라인	점수를 대응 단계(1~5)로 변환하는 기준
 weight-set	과목별 가중치	여러 사실 데이터를 점수로 합치는 배합비
 run-context	디지털 영수증	"왜 이 결과가 나왔지?" 사후 추적의 출발점

3. source — 29건의 데이터 카탈로그

source 는 데이터 자체가 아니라 "데이터 종류 1개의 메타정보" 다.

 source 노드 1개 = "데이터 종류의 카탈로그"

source-id

"SOURCE_KMA_OBSERVED_WEATHER"

source-name

"기상 관측/강수 관측"

availability-status

"REAL"

비유: 마트 영수증이 아니라 장보기 리스트의 한 줄.

3종 status 의 의미

 REAL

"진짜 받을 수 있다"

18건

공공데이터/국가 시스템

 MOCK

"지금은 가짜로 채움"

6건

담당 부서 시스템 미연계
→ LLM 인용 시 "추정값" advisory 자동 부착

 EXCLUDE

"이번 스터디 범위 밖"

5건

대체 가능 / 보안 / 범위 밖
→ 의사결정에 영향 없음

Σ 29

MOCK 6건의 정체 (공통점: 현장 운영 데이터)

1. 도로 진입 가능성 / 2. 소화전·비상소화장치 / 3. 지표연료 상태 (낙엽·마른풀) /
2. 작업장 위험 / 5. 소방 자원 가용성 / 6. 예비주수 장비 능력
- 운영 전환 체크리스트: 이 6건을 실제 source 로 갈아끼우면 운영 단계로 전환.

4. segment — 종합 차트 (입력 + 추론 결과)

segment 5건이 적재돼 있다. 각 노드는 **입력 attribute** 와 **추론 결과 attribute (write-back)** 를 같이 가진다.

[입력]

segment-id, segment-name, admin-region
risk-grade, alert-level
residential-population, forest-distance-m
wind-speed, safety-class

[추론 결과 — write-back]

s-priority (점수, 0~1)
base-state (점수만으로 결정한 단계)
state (override 적용 후 최종 단계)
state-override-reason (왜 override 됐는지)

5건 segment 의 추론 결과

동네	점수	base-state	최종 state	비고
여수 상암동	0.534	Review	Priority ↑	경보 override
장흥 유치면	0.454	Review	NotActionable 🛑	안전 차단
광주 동구 (사례)	0.180	General	General	변화 없음
무안 운남면	0.295	Enhanced	Enhanced	변화 없음
나주 봉황면 ★	0.350	Enhanced	Enhanced	v2 정책이면 격상

5. threshold-set — 정책 카드 (코드 0줄 시뮬레이션의 핵심)

 threshold-set v1 (현행)

th-low0.20

th-mid-low0.40

th-mid-high0.60

th-high0.80

v2 (보수적)

0.15

0.30

0.50

0.70

v2 가 "보수적"인 이유: 경계선이 왼쪽 이동 → 동일 점수가 더 높은 단계로 격상됨.

v1 → v2 시뮬레이션 결과 (실제 적재 기준)

동네	점수	v1 정책	v2 정책	결과
여수 상암동	0.534	Review	Priority	격상 (정보 override 유지)
★ 나주 봉황면	0.350	Enhanced	Review	한 단계 격상! ↑
무안 운남면	0.295	Enhanced	Enhanced	변화 없음

🎯 **포인트:** 코드를 한 줄도 안 고치고 INSERT 한 줄로 정책 v2 를 추가했더니, 나주 봉황면이 즉시 격상 됨. 데이터가 곧 정책 인 구조의 실증.

6. State 6종 — 산불 대응 단계 사전

점수 낮음 → 점수 높음		
① GeneralManagement	평상 관리	● 일상 순찰
② EnhancedMonitoring	감시 강화	● 모니터링 ↑
③ ReviewPreWatering	예비 주수 검토	● 회의 소집
④ PriorityPreWatering	우선 예비 주수	● 자원 배정
⑤ ImmediatePreWatering	즉시 예비 주수	🔥 출동·살수
□ NotActionable	작업 불가	🚫 점수 무관, 안전 강제 차단

예비 주수(): 산불 발생 전 위험 지역에 미리 물을 뿌리는 예방 조치. 의사결정의 핵심 액션.

7. 왜 이 5종 구조인가

이 구조의 핵심은 "데이터가 추론을 낳고, 추론이 다시 데이터가 되어(write-back), 모든 과정이 영수증(run-context)으로 남는" 투명한 의사결정 체계다.

특히 threshold-set 과 weight-set 을 **별도 노드** 로 분리한 게 결정적. 이게 코드 안에 상수로 박혀 있었다면, 정책을 바꿀 때 코드 배포가 필요했을 것이다. 노드로 빠져 있기 때문에 INSERT 한 줄로 정책 시뮬레이션이 가능하다.



한 줄 요약

"source(원료) + weight·threshold(레시피) → segment(요리) → run-context(영수증). 레시피를 노드로 빼낸 게 코드 0줄 시뮬레이션의 핵심."

3. 추론 과 LLM 연동

데이터에서 결과가 나오는 메커니즘. TypeQL `fun` 의 핵심 트릭 + Python ↔ TypeQL Write-back + LLM 질의 흐름.

1. TypeQL — 스키마 / fun / 데이터 3단 구조

스키마 (데이터 카드의 설계도)

```
define
  threshold-set sub entity,
    has config-version,    # v1, v2 등 정책 버전
    has th-low,            # 단계별 경계값 (예: 0.20)
    has th-mid-low,
    has th-mid-high,
    has th-high;
```

fun (DB 내부에 저장된 판단 로직)

```
define
  fun s_priority_to_state($sp, $version) -> { string }:
    match
      # 1. 정책 버전($version)에 해당하는 노드를 DB에서 조회
      $t isa threshold-set, has config-version $version;
      $t has th-low $tl, has th-mid-low $tml,
        has th-mid-high $tmh, has th-high $th;

      # 2. 동적으로 조회한 경계값과 점수 비교
      if ($sp < $tl) { fetch { "GeneralManagement" }; }
      else if ($sp < $tml) { fetch { "EnhancedMonitoring" }; }
      else if ($sp < $tmh) { fetch { "ReviewPreWatering" }; }
      else if ($sp < $th) { fetch { "PriorityPreWatering" }; }
      else { fetch { "ImmediatePreWatering" }; }
```

🎯 **핵심 트릭:** 함수 내부에 임계값 숫자를 하드코딩하지 않고, `match` 문으로 DB 내 정책 노드를 동적으로 조회한다.

데이터 (실제 정책 카드)

```
insert
  $v1 isa threshold-set, has config-version "v1",
    has th-low 0.20, has th-mid-low 0.40,
    has th-mid-high 0.60, has th-high 0.80;

  $v2 isa threshold-set, has config-version "v2",
    has th-low 0.15, has th-mid-low 0.30,
    has th-mid-high 0.50, has th-high 0.70;
```

→ 새 정책 v3 를 추가하려면: 코드 0줄, INSERT 한 줄. 함수는 매개변수만 바꾸면 즉시 새 정책으로 동작.

2. Python → TypeQL Write-back 4단계

추론 결과가 어떻게 "데이터" 가 되는가 — 4단계.

```
[Step 1. 조회] Python → TypeQL (match/fetch)
"여수 상암동의 인구·바람 알려줘"
→ population=8000, wind_speed=16.0

[Step 2. 계산] Python (formulas.py)
score = calculate_priority(8000, 16.0)
→ 0.560 (예시)

[Step 3. 쿼리 생성] Python → TypeQL (insert)
query = f"match $s isa segment, has segment-id 'EMD_여수_상암동';
insert $s has s-priority {0.560};"

[Step 4. 기록] DB
(Before) segment — has segment-id "EMD_여수_상암동"
(After)  segment — has segment-id "EMD_여수_상암동"
                    └─ has s-priority 0.560 ← NEW 영구 추가
```

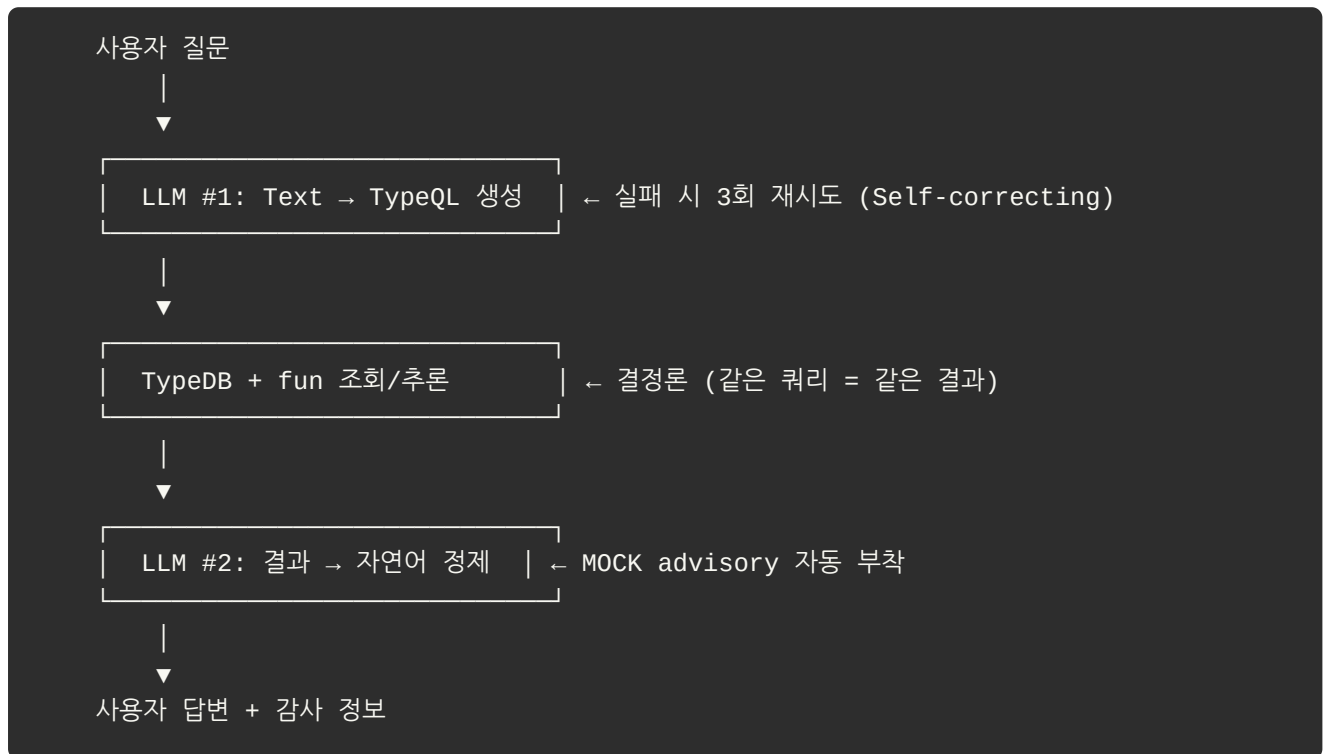
역할 분담

단계	주체	동작
1. 조회	Python → TypeQL	DB 의 기초 사실 읽기
2. 계산	Python	메모리 상에서 가중치 산식
3. 생성	Python → TypeQL	insert 문 동적 조립
4. 기록	TypeQL (DB)	노드에 attribute 로 영구 저장

Python = 두뇌, TypeQL = 손. 두뇌의 생각을 DB 에 도장 찍는 게 TypeQL.

3. 사용자 질의 흐름 (LLM #1 + DB + LLM #2)

자연어 질문이 답변으로 나오는 흐름.



두 LLM 호출의 역할 차이

	LLM #1	LLM #2
입력	사용자의 자연어 질문	DB 조회 결과 (JSON)
출력	TypeQL 쿼리 문자열	자연어 답변
실패 시	3회까지 재시도 → golden fallback	정제 실패 시 raw JSON 노출
역할	사용자 의도 → 그래프 언어	그래프 결과 → 사람 언어

사실 생성은 DB+fun, 표현 변환만 LLM. 환각이 끼어들 자리가 없다.

4. 환각 방지의 두 축

축 1 — MOCK advisory

LLM 이 MOCK 데이터를 인용해 답할 때, 응답에 "이건 추정값입니다" 가 자동으로 따라붙는다. source 의 `availability-status` 가 답변까지 따라가는 구조.

축 2 — 데이터 부재 시 거부

DB 에 없는 데이터를 LLM 이 추측해서 답하지 않는다. 명확히 거부한다.

Q: "1990년 1월 광주 동구 위험도?"
A: "해당 시점의 데이터가 존재하지 않습니다. (실패 처리 - 추측하지 않음)"

→ 사실 생성을 LLM 에서 떼어낸 결과.

5. 응답 JSON = 모든 출력의 단일 원천

추론 결과 JSON 1개가 본 시스템의 세 출력으로 흘러간다.



- V-World GIS: `state` 별 색상 히트맵
- Superset 표: `s_priority` 랭킹표
- 감사 로그: `run_context_id` + `git_commit` 으로 "어느 시점, 어떤 코드로 나온 결과인가" 추적

6. 왜 이 구조인가 — 사실 vs 표현의 분리

전통적인 LLM 시스템: LLM 이 사실도 만들고 표현도 한다 → 환각 발생.

본 시스템: 사실은 그래프와 함수가, 표현은 LLM 이. 두 책임을 분리하면 환각 영역이 차단된다.

책임	담당	특징
사실 생성	TypeDB + fun (Python 산식)	결정론, 재현 가능, 추적 가능
표현 변환	LLM (Gemini → 운영 시 내부 LLM)	자연어로의 변환만

→ 이렇게 책임을 분리한 게 본 시스템의 환각 방지·감사 추적·모듈성 설계의 핵심이다.

 한 줄 요약

"fun = DB 안의 판단 로직 + 정책 노드 동적 조회. Python(두뇌) ↔ TypeQL(손) Write-back. LLM 은 표현만 담당해 환각 차단."

4. 시연 4개 요약

발표 25분 동안 실행된 4개 시연의 화면별 가이드. 실행 명령 + 화면 + 메시지를 시연당 1쪽으로 압축.

시연 1 — write-back

실행 명령

```
$ ./run.sh db/check.py
$ ./run.sh inference/run_inference_demo.py
```

출력 화면

💡 **화면 약어:** EMD_ = 읍·면·동 행정구역 (예: EMD_여수_상암동 = 여수시 상암동),
S= = 종합 위험 점수 (0~1)

```
$ ./run.sh db/check.py

[검증] source 적재: REAL 18 / MOCK 6 / EXCLUDE 5
[검증] segment 적재: 5건

$ ./run.sh inference/run_inference_demo.py

[추론] 5 segment 점수+State 산출 → 그래프 write-back

EMD_여수_상암동    S=0.534    PriorityPreWatering
EMD_강흥_유치면    S=0.454    NotActionable
EMD_나주_봉황면    S=0.315    EnhancedMonitoring
EMD_무안_운남면    S=0.315    EnhancedMonitoring
EMD_광주_동구      S=0.180    GeneralManagement
```

✅ 그래프에 저장 완료

핵심 메시지

4주차 source 29건 위에 segment 5건이 올라가고, Python 이 그래프의 임계값을 읽어 점수를 계산한 다음 그 결과를 그래프에 **다시 써넣었다**. 다음 시연부터는 전부 이 저장된 결과를 조회한다.

🔥 메시지 ①: "추론 결과가 다시 데이터가 된다 (write-back)"

시연 2 — fun 듀얼 ★

실행 명령 (터미널 2개 분할)

```
[왼쪽] $ ./run.sh tests/test_param_externalization.py
[오른쪽] $ ./run.sh tests/test_python_vs_fun.py
```

출력 화면

[왼쪽: 정책 시뮬레이션]

```
v1 임계값: 0.20/0.40/0.60/0.80
v2 임계값: 0.15/0.30/0.50/0.70

★ EMD_나주_붕항면 0.315
  v1: 감시 강화 → v2: 검토 격상 ↑
→ 코드 수정 0줄, INSERT로 시뮬레이션
```

[오른쪽: 회귀 검증]

```
회귀 검증 — Python ↔ fun 일치

[v1 임계값] 5/5 일치 ✓
[v2 임계값] 5/5 일치 ✓

→ 10/10 통과
"추론을 fun으로 옮겨도 결과 동일"
```

핵심 메시지

왼쪽은 **코드 수정 없이 정책 노드만 바꿔** 시뮬레이션한 결과. 나주 붕항면이 격상됐다.

오른쪽이 진짜 새로운 부분 — 같은 추론을 TypeDB **fun** 이 **직접 수행**했는데 결과가 100% 일치. 즉 추론을 그래프 안으로 완전히 옮길 수 있다.

🔥 메시지 ②: "정책 변경 = INSERT 한 줄, 코드 0줄"

🔥 메시지 ⑥: ★ "그래프가 직접 추론할 수 있다"

시연 3 — chat.py + advisory

실행 명령

```
$ ./run.sh llm/chat.py
```

출력 화면 (질의 3건)

Q1: EMD_여수_상암동 위험도 알려줘

💬 답변: "여수 상암동은 현재 PriorityPreWatering(우선 예비주수) 상태입니다.
기상특보 발효로 인해 단계가 격상되었습니다."

Q2: MOCK source 모두 알려줘

💬 답변: "[리스트 나열]... ⚠️ 일부 데이터는 추정값(MOCK)입니다."
(advisory 자동 부착)

Q3: 1990년 1월 광주 동구 위험도?

💬 답변: "해당 시점의 데이터가 존재하지 않습니다."
(실패 처리 - 추측하지 않음)"

핵심 메시지

Q1 — 일반 질의. DB 조회 결과를 자연어로 풀어준다.

Q2 — MOCK 인용 시 **advisory 자동 부착**. source.availability-status 가 답변까지 따라가는 구조.

Q3 — 환각 방지의 본질. **DB 에 없는 데이터를 LLM 이 추측해 답하지 않는다**. 명확히 거부한다.

🔴 메시지 ③: "감사 정보가 자연어에 묻어 나온다"

🔴 메시지 ④: "DB 에 없으면 LLM 도 답을 만들지 않는다"

시연 4 — 평가셋 14건

실행 명령

```
$ ./run.sh llm/eval/run_eval.py
```


출력 화면 (실시간 카운트)

평가셋 14건 자동 실행 중...

[3/14] 단순

↺ 재시도 1회 → 성공

← "자가 수정!"

[11/14] 복잡

↺ 재시도 1회 → 성공

[12/14] 복잡

✖ 1990년 데이터 없음 (의도된 거부)

✓ 직접 성공

8건

↺ 자가 수정

3건 ← 핵심 관찰 포인트

↺ Fallback

0건

✖ 완전 실패

3건 ← 1건은 환각 방지용 데이터 부재

직접 처리율: 11/14 (79%)

핵심 메시지

↺ 표시가 뜰 때마다 LLM 이 한 번 틀렸다가 시스템이 다시 시도해서 성공한 것이다. 이게 Self-correcting 이다.

✖ 는 데이터가 없는 경우로, 시스템이 거짓말을 하지 않고 정확히 거절한 결과다.

🔥 메시지 ⑤: "자가 수정 + 정량 측정 = 측정 가능한 LLM 시스템"

6대 Takeaway 총정리

#	메시지	시연
①	write-back	1
②	외부화 (코드 0출)	2
③	advisory	3
④	환각 방지	3·4
⑤	Self-correcting	4
⑥	★ 자족 추론	2

4개 시연이 함께 증명하는 것

"사실은 그래프와 함수가, 표현은 LLM 이. 정책은 노드로 외부화. 측정·자가수정은 시스템이."

- 시연 1:2 → "사실은 그래프와 함수가" 부분 증명
- 시연 3 → "표현은 LLM 이 + 정책은 노드로" 부분 증명
- 시연 4 → "측정·자가수정은 시스템이" 부분 증명

시연을 직접 재현하고 싶다면

받으신 코드 zip 안에 모든 시연이 들어 있다. 환경 셋업 + 명령 시퀀스는 [5_코드_안내.md] 참고.

한 줄 요약

"4개 시연 = 4개 실행 명령 + 4개 출력 화면 + 6개 메시지. 한 가설(사실/표현 분리)의 4각도 증거."

5. 코드 안내 — 받으신 코드 zip 따라가기

이 자료와 함께 전달된 코드 zip (study/wildfire-poc/) 의 구조와 읽는 순서.

1. 전체 폴더 트리

study/wildfire-poc/	
├─ README.md	← 환경 셋업
├─ run.sh	← Python 스크립트 wrapper
├─ db/	← 🗄️ DB 계층
│ ├─ schema_v2.tql	← 5종 엔티티 정의 (현행)
│ ├─ functions.tql	← fun (5개) – 자족 추론
│ ├─ apply_schema_v2.py	← 스키마 적용
│ ├─ apply_functions.py	← 함수 적용
│ ├─ seed_params.py	← threshold v1/v2 + weight 적재
│ ├─ load_sources.py	← source 29건 적재
│ ├─ load_segments.py	← segment 5건 적재
│ └─ check.py / check_all.py	← 조회 도구
├─ inference/	← 📊 추론 계층
│ ├─ formulas.py	← 점수 산식 (Python)
│ ├─ graph_inference.py	← Python 추론 (As-is)
│ ├─ graph_inference_fun.py	← fun 추론 (To-be) ★
│ └─ run_inference_demo.py	← 추론 실행 + write-back
├─ llm/	← 💬 LLM 계층
│ ├─ text_to_typeql.py	← 자연어 → TypeQL (Gemini)
│ ├─ chat.py	← 대화 + advisory
│ └─ eval/run_eval.py	← 평가셋 14건 자동 실행
├─ demo/	← 🌐 데모 웹 (FastAPI, :8000)
│ ├─ server.py	
│ ├─ api/demo1.py ~ demo4.py	← 시연 ①~④ API
│ └─ run_demo.sh	
├─ tests/	← ✅ 회귀 테스트
│ ├─ test_python_vs_fun.py	← Python ↔ fun 일치 검증
│ ├─ test_param_externalization.py	← v1↔v2 정책 전환 검증
│ └─ test_golden_graph.py	
├─ _spike/	← 🖋️ 실험 (TypeQL fun 검증)
│ └─ 01~02d 단계별 spike	
└─ scripts/	← 🛠️ 유틸 (DB 리셋 등)

포트 메모: TypeDB 는 localhost:1729 , 데모 웹은 localhost:8000 — 다른 포트.

2. 계층별 책임

계층	폴더	책임
DB	db/	스키마·함수 정의, 데이터 적재, 조회
추론	inference/	점수 계산, 그래프 write-back
LLM	llm/	자연어 ↔ TypeQL, 평가
데모	demo/	FastAPI 웹 시연
테스트	tests/	회귀 검증
실험	_spike/	TypeDB 3.0 fun 도입 전 검증 (참고용)

3. 환경 셋업

Docker — TypeDB 기동

```
docker run -d --name typedb -p 1729:1729 typedb/typedb:3.4.0
```

Python 환경

```
cd study/wildfire-poc
uv sync # 의존성 설치
```

API 키 설정

```
export GEMINI_API_KEY="your-key-here"
```

초기 데이터 적재 (한 번만)

```
./run.sh db/apply_schema_v2.py
./run.sh db/apply_functions.py
./run.sh db/seed_params.py
./run.sh db/load_sources.py
./run.sh db/load_segments.py
```

4. 코드 읽는 순서 가이드

코드를 처음 접하는 사람에게 권장 순서:

- | | |
|-----------------------------------|-------------------------|
| ① README.md | 환경 셋업 |
| ▼ | |
| ② db/schema_v2.tql | "어떤 데이터가 있는가" (5종 엔티티) |
| ▼ | |
| ③ db/functions.tql | "어떻게 추론하는가" (fun) |
| ▼ | |
| ④ db/seed_params.py | "어떤 정책 카드가 있는가" (v1/v2) |
| ▼ | |
| ⑤ inference/formulas.py | "점수가 어떻게 계산되는가" |
| ▼ | |
| ⑥ inference/graph_inference.py | "Python 추론 어떻게 도는가" |
| ▼ | |
| ⑦ inference/run_inference_demo.py | "실제 실행 + write-back" |
| ▼ | |
| ⑧ llm/chat.py | "사용자 질의가 어떻게 처리되는가" |
| ▼ | |
| ⑨ tests/test_python_vs_fun.py | "Python ↔ fun 결과 일치 검증" |

5. 시연 ↔ 코드 매핑

시연	실행 명령	호출되는 파일
시연 1 (write-back)	<code>run.sh db/check.py</code> <code>run.sh inference/run_inference_demo.py</code>	<code>db/check.py</code> , <code>formulas.py</code> , <code>graph_inference.py</code>
시연 2 (fun 듀얼)	<code>run.sh tests/test_param_externalization.py</code> <code>run.sh tests/test_python_vs_fun.py</code>	<code>seed_params.py</code> , <code>graph_inference.py</code> , <code>graph_inference_fun.py</code>
시연 3 (chat + advisory)	<code>run.sh llm/chat.py</code>	<code>chat.py</code> , <code>text_to_typeql.py</code>
시연 4 (평가셋 14건)	<code>run.sh llm/eval/run_eval.py</code>	위 모두 + 평가셋

데모 웹으로 보고 싶다면

```
./demo/run_demo.sh
# → http://localhost:8000 접속
```

시연	데모 API
①	<code>demo/api/demo1.py</code>
②	<code>demo/api/demo2.py</code>
③	<code>demo/api/demo3.py</code>
④	<code>demo/api/demo4.py</code>

6. 왜 이 구조인가 — "사실 vs 표현 분리" 의 코드 구현

계층	의미
db/	사실의 보관소 (스키마 + 데이터 + fun)
inference/	사실을 사실로 변환 (점수 계산 + write-back)
llm/	사실을 표현으로 변환 (자연어 답변)
demo/	표현의 시각화 (웹 UI)
tests/	사실 일관성 검증

각 계층은 독립적으로 교체 가능:

- LLM 모델 교체 → llm/text_to_typeql.py 만 수정
- 추론을 Python → fun 으로 완전 이전 → inference/graph_inference_fun.py 로 스위치
- 이게 본 시스템의 모듈성 설계의 핵심 이다.

7. 환경 트러블슈팅 (자주 나오는 이슈)

증상	원인 / 대처
TYPEDB_UNAVAILABLE 에러	TypeDB 컨테이너 미기동 → docker ps 확인, docker restart typedb
GEMINI_API_KEY 없음	환경변수 미설정 → export GEMINI_API_KEY=...
segment 조회 결과 없음	초기 적재 미실행 → § 3 의 "초기 데이터 적재" 5줄 재실행
포트 1729 충돌	다른 컨테이너 실행 중 → docker stop \$(docker ps -q --filter publish=1729)

💡 한 줄 요약

"5계층(DB·추론·LLM·데모·테스트) + 1실험(_spike). TypeQL 은 db/ 안에만, 나머지는 Python. run.sh 가 모든 입구."